

七星微赛题初赛设计说明书

面向七星微赛题的工程化 RISC-V 处理器方案

参赛作品：YH_rv_cpu

项目名称：YH_rv_cpu

报告版本：v1.1

更新日期：2026-04-17

冻结主口径：RV32I + Zicsr

目标原型平台：Nexys A7-100T

文档性质：初赛正式设计说明书

摘要

YH_rv_cpu 是一套面向七星微赛题的工程化 RISC-V 处理器方案。该设计以 RV32I + Zicsr 为冻结性能与提交主口径，采用 IF/ID/EX/MEM/WB 五级顺序流水，并构建了由同步指令存储、数据存储、UART、timer 和 DONE 标志组成的最小 SoC，使 RTL、软件、回归验证、CoreMark 和 FPGA 原型路径能够保持一致。

当前工程已经形成覆盖功能正确性、性能评估和原型实现可行性的完整证据链。功能验证方面，除基础 smoke 回归外，还完成了 rv32 baseline 33/33、rv64 baseline 21/21、rv32 full-ui 42/42 和 rv64 full-ui 54/54。性能方面，CoreMark short 为 0.912472 CoreMark/MHz，strict 长跑为 0.912465 CoreMark/MHz 且运行时间 10.959325s。原型实现方面，Vivado impl50 结果为 2556 LUT / 2170 FF / 4 BRAM / 0 DSP，并保持 WNS = +5.599ns、WHS = +0.025ns 的正时序裕量。

综合来看，该设计已经具备清晰的微架构边界、可复核的验证矩阵、稳定的性能记录以及明确的 FPGA 原型适配路径。同时，当前工程还具备 RV32/RV64 双 XLEN 验证能力，为后续实体板 bring-up、控制流优化和系统扩展保留了清晰起点。

关键词

RISC-V；五级流水；工程化验证；CoreMark；FPGA 原型；双 XLEN

重要内容预览

1. **CPU 架构设计说明书**: 正文给出五级流水 CPU、最小 SoC、冒险处理、CSR/trap/timer 支持与模块职责划分，重点说明数据流、控制流与参数化边界。
2. **Verilog/VHDL 建模规范**: 正文单列 HDL 建模规范章节，说明模块划分、组合/时序逻辑约束、参数化策略、接口组织方式与测试平台协同规则。
3. **验证计划与测试报告**: 正文给出 smoke、baseline、扩展 full-ui、CoreMark、Vivado impl50 与 FPGA-like probe 的验证矩阵和主要结果。
4. **FPGA 适配指南**: 正文说明目标板卡、固定原型参数、综合实现口径、报告位置以及后续 bring-up 路径，满足赛题中对 FPGA 原型适配材料的结构要求。

目录

摘要	1
关键词	1
重要内容预览	1
1 项目概述与设计目标	4
1.1 赛题要求对齐	4
1.2 设计目标	4
2 CPU 架构设计说明书	6
2.1 系统总体架构	6
2.2 CPU 五级流水组织	7
2.3 数据相关处理	8
2.4 控制相关与 redirect 机制	8
2.5 CSR、异常与中断处理	9
2.6 SoC 存储与 MMIO 组织	9
2.7 参数化与扩展验证支撑	9
2.8 架构取舍	10
3 Verilog/VHDL 建模规范	10
3.1 建模口径说明	10
3.2 模块划分规则	10
3.3 组合逻辑与时序逻辑规则	11
3.4 参数化与可扩展性规则	12
3.5 接口与命名规范	12
3.6 测试平台协同规范	12
3.7 本节结论	13
4 验证计划与测试报告	13
4.1 验证矩阵	13
4.2 验证口径说明	14
4.3 基础回归与 ISA 验证结果	14
4.4 CoreMark 性能结果	15
4.5 FPGA 实现结果	16
4.6 验证结论	16

5	FPGA 适配指南	16
5.1	原型目标与实现路径	16
5.2	原型路径固定配置	17
5.3	关键结果	18
5.4	实体板相关工作	18
5.5	小结	18
5.6	当前可提交的适配材料	18
6	结论与后续工作	19
6.1	综合结论	19
6.2	当前边界	19
6.3	后续工作	19
6.4	结语	20
A	关键结果与证据索引	21

1 项目概述与设计目标

1.1 赛题要求对齐

结合赛题题面要求，本项目的说明书需要覆盖以下四类正式内容：

- CPU 架构设计说明书；
- Verilog/VHDL 建模规范；
- 验证计划与测试报告；
- FPGA 适配指南。

围绕上述要求，本设计采用五级顺序流水结构，以 RV32I + Zicsr 为核心冻结子集，围绕最小 SoC 和 FPGA 原型实现建立完整工程链路。说明书编排上不再采用泛化的工程周报式结构，而是按赛题要求把“架构说明、建模规范、验证报告、FPGA 适配”作为正文主线，便于评审直接对照核查。

1.2 设计目标

本项目在初赛阶段设置了五项核心目标：

1. **架构清晰**：构建层次分明、职责明确的 CPU 与 SoC 结构，便于说明微架构原理；
2. **验证充分**：建立从 smoke、baseline、扩展 ISA 覆盖到 CoreMark 和实现结果的验证矩阵；
3. **工程可复现**：使关键脚本、文档、RTL、测试平台和结果摘要保持一致；
4. **原型可延伸**：在保持实现约束可控的前提下，为 FPGA 原型和后续性能优化保留清晰路径；
5. **扩展有边界**：在保持冻结提交口径稳定的同时，保留 RV32/RV64 双 XLEN 验证和后续优化接口。

表 1: 工程特征与设计边界

主题	当前口径
冻结主口径	采用 RV32I + Zicsr 五级顺序流水，作为当前对外性能与提交主叙述路径
扩展验证能力	当前工程验证已覆盖 RV32/RV64 双 XLEN，并具备 baseline 与更广 full-ui 覆盖
SoC 组织	采用同步指令存储、数据存储与最小 MMIO 外设构成闭环，优先保证仿真、回归与 FPGA 路径一致性
机器态支持	当前重点覆盖 CSR、trap、mret 和 timer interrupt 等比赛阶段确有需求的机器态能力
原型实现目标	以 Nexys A7-100T 为目标平台，形成 impl50、报告与 bring-up 路径的统一工程入口
非当前目标	暂不引入 cache、多发射、乱序或最终板级 signoff 等高风险复杂结构

表 2: 设计范围概览

设计内容	方案说明
处理器内核	采用 IF/ID/EX/MEM/WB 五级顺序流水，覆盖 hazard、redirect、CSR 与 trap 关键机制
最小 SoC	由同步指令存储、数据存储、UART、timer 和 DONE 构成，用于承接软件运行与对外观测
功能验证	给出 smoke、rv32 baseline 33/33、rv64 baseline 21/21 以及扩展 full-ui 验证结果
性能评估	给出 CoreMark short、strict >=10s 与 FPGA-like probe 的结果，用于说明性能稳定性与路径一致性
FPGA 原型适配	给出 Vivado impl50 的资源、时序和原型路径固定配置结果，用于说明原型实现可行性
后续工作	实体板连线验证、现场观测和最终板级 I/O 约束确认需在外部条件具备后继续开展

本文重点说明以 RV32I + Zicsr 为冻结性能口径的处理器及其最小 SoC 实现；同时给出 RV32/RV64 扩展 full-ui 与 baseline 结果，用于说明该工程已具备双 XLEN 验证能力与更广测试范围下的稳定性。

2 CPU 架构设计说明书

2.1 系统总体架构

YH_rv_cpu 采用“CPU 核 + SoC 封装 + FPGA 原型顶层”的层次化组织方式。CPU 内核负责指令执行、控制流处理和异常响应；SoC 层负责连接同步指令存储、数据存储和最小 MMIO 外设；FPGA 顶层负责将工程映射到 Nexys A7-100T 的原型实现路径。

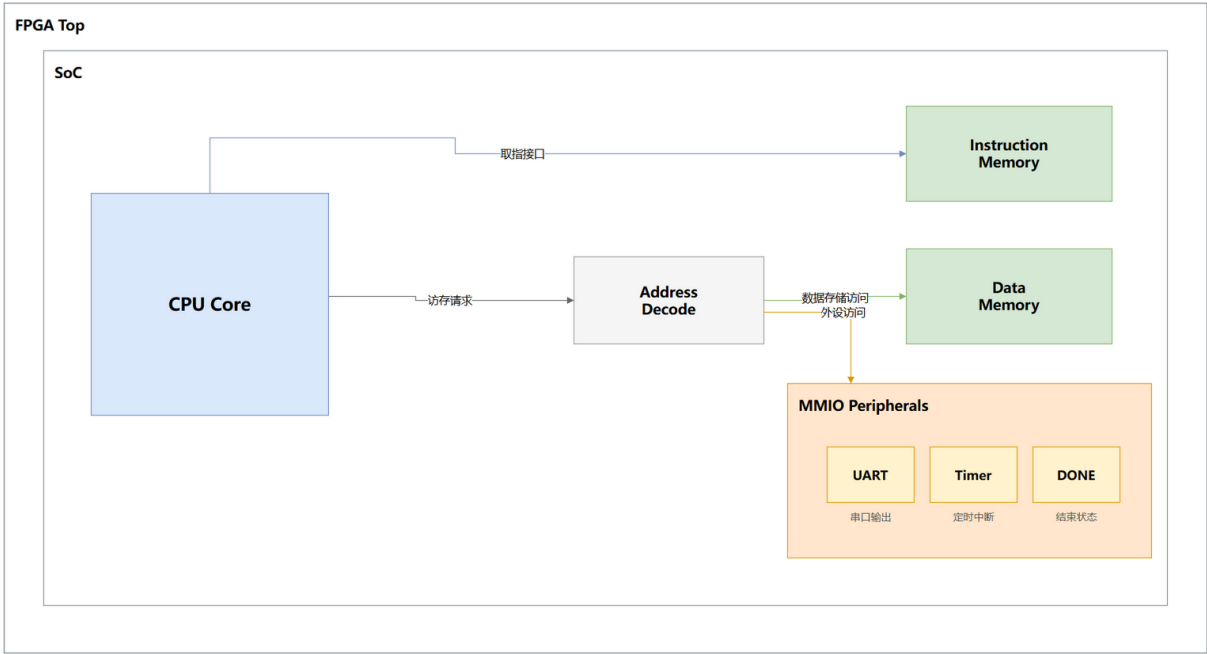


图 1：系统总体架构与关键模块关系图

图 1 强调了三层边界关系：CPU 内核承接执行语义，SoC 层负责可运行的软件系统环境，FPGA Top 负责将同一套工程映射到原型平台。这种组织方式避免了“单独为说明书维护一套结构、单独为原型实现再维护另一套结构”的割裂问题，有利于保证设计说明、RTL、验证和实现报告之间的一致性。

表 3: 核心模块职责

模块	作用
YH_rv_cpu	CPU 顶层，组织 IF/ID/EX/MEM/WB 五级流水、前递、redirect、CSR 与 trap
YH_rv_cpu_hazard_unit	负责 load-use hazard 检测和前递选择，是停顿控制的关键模块
YH_rv_cpu_soc	SoC 封装层，连接同步指令存储、数据存储与最小 MMIO 外设
YH_rv_sync_imem_rom	服务同步取指路径，使 CPU 与 FPGA 原型实现保持一致
YH_rv_dmem_ram	负责 load/store 数据访问，并与 MMIO 共同组成访问路径
YH_rv_cpu_fpga_top	FPGA 原型顶层与综合参数入口，用于承接 impl50 等实现流程

2.2 CPU 五级流水组织

处理器采用经典五级流水：

- 1. IF：同步取指、PC 更新与 redirect 接收；
- 2. ID：译码、寄存器读、立即数生成和控制信号准备；
- 3. EX：ALU 运算、分支跳转判断、CSR 访问以及 trap/mret 决策；
- 4. MEM：load/store 与 MMIO 访问；
- 5. WB：将 ALU、访存或 CSR 结果回写寄存器堆。

这种结构的优势在于边界清楚、验证成本可控，并且足以支撑 CoreMark、riscv-tests 和最小 SoC 软件运行。初赛版本没有引入 cache、多发射或乱序机制，而是优先保证结构可说明、行为可验证、结果可复现。

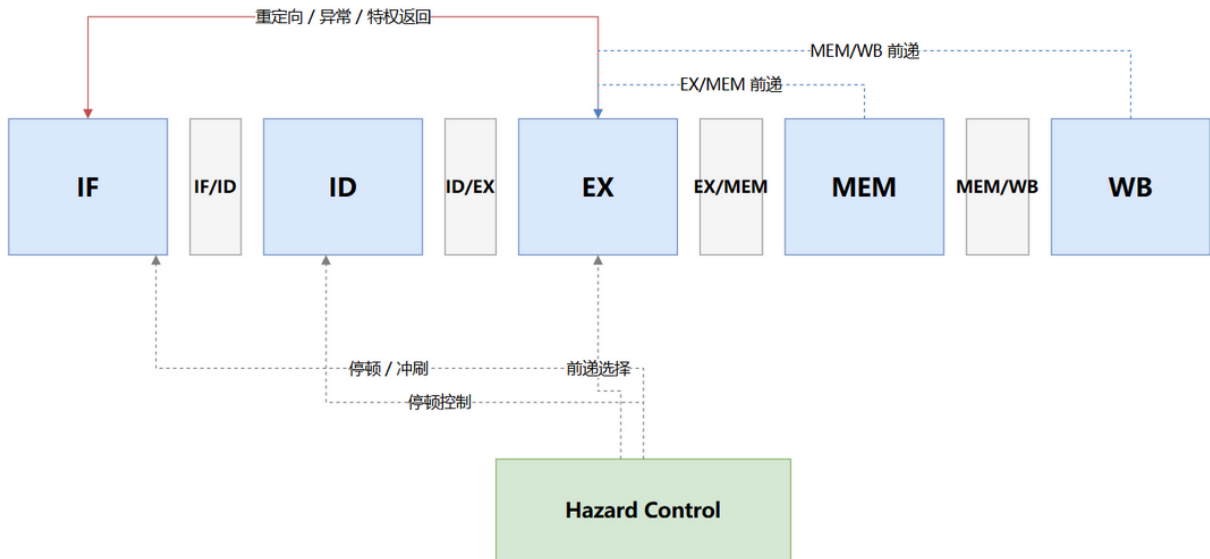


图 2: 五级流水与关键控制通路示意图

图 2 突出展示了两类关键通路：一类是 EX 级返回 IF 的控制流重定向路径，用于统一处理 branch、jump、trap 和 mret；另一类是围绕 EX 的前递和停顿控制路径，用于在保持顺序流水结构清晰的前提下收敛性能损失。

2.3 数据相关处理

为了在顺序流水中兼顾正确性和吞吐，项目实现了明确的数据相关处理策略：

- EX/MEM、MEM/WB 两级前递，降低普通 ALU 相关导致的停顿；
- 针对 load-use 场景设置专门检测逻辑，当结果尚未可用且后继指令立即消费时暂停译码级；
- 译码级停顿条件采用 `stall_decode = load_use_hazard` 策略，使停顿范围尽量收敛在必要场景内。

这一选择的直接效果是：对可前递场景尽量放行，对不可前递的 load-use 场景执行最小必要停顿，在结构复杂度、稳定性和性能之间取得平衡。

2.4 控制相关与 redirect 机制

控制流处理主要在 EX 级完成决策：

- branch、jump 和 jalr 在 EX 级形成 redirect；
- trap、mret 与普通跳转共享控制流重定向框架；
- 当 `ex_fetch_redirect_valid` 有效时，IF 级更新 PC 并清空错误路径气泡。

这种组织方式能够用统一框架处理普通跳转、异常进入和异常返回，既保持控制路径简洁，也便于验证 trap 与正常执行之间的切换关系。

2.5 CSR、异常与中断处理

本设计实现的控制状态寄存器与异常处理，重点覆盖比赛阶段确有需求的机器态功能：

- CSR 指令：csrrw/csrrs/csrrc 及其立即数变体；
- 关键机器态 CSR：mstatus、mie、mtvec、mscratch、mepc、mcause、mip；
- 同步异常：非法指令、CSR 非法访问、load misaligned 等；
- 控制流相关：ecall、ebreak、mret；
- 机器定时器中断：通过 timer 外设触发并进入 trap 流程。

这一范围能够满足现阶段软件运行和验证需要，也有利于将实现重点集中在基础执行链路、异常响应和工程完整性上。

2.6 SoC 存储与 MMIO 组织

SoC 采用“ROM + RAM + MMIO”的最小闭环结构。其中同步 ROM 负责取指，RAM 负责数据访问，MMIO 外设负责提供对外可观测接口和计时能力。关键地址组织如下：

表 4: SoC 关键地址映射

资源	地址/基址	作用
ROM	0x0000_0000	程序存储基址
RAM	0x0000_4000	数据存储基址
UART TX	0x1000_0000	串口发送寄存器
DONE	0x1000_0004	程序结束标志寄存器
TIMER VALUE LO	0x1000_0008	计时值低 32 位
TIMER VALUE HI	0x1000_000C	计时值高 32 位
TIMER CMP LO	0x1000_0010	比较寄存器低 32 位
TIMER CMP HI	0x1000_0014	比较寄存器高 32 位
TIMER CTRL	0x1000_0018	定时器控制与中断使能

这种组织方式具有三方面优势：一是能够直接支撑仿真、CoreMark 和 riscv-tests；二是向 FPGA 原型迁移成本较低；三是 UART、timer 和 DONE 为后续 bring-up 提供了最小但有效的观测接口。

2.7 参数化与扩展验证支撑

虽然冻结提交口径以 RV32I + Zicsr 为主，但当前工程并没有把结构边界固化为单一 XLEN。CPU 顶层、测试脚本和回归清单已经具备参数化与多清单组织能力，使项目能够

在保持提交主口径稳定的同时，完成 RV32/RV64 双 XLEN baseline 与扩展 full-ui 验证。这样的工程组织有两项直接收益：

- 一方面，它说明当前设计不是“只能跑单一路径”的一次性实验实现，而是具备继续扩展和继续验证的基础；
- 另一方面，它使冻结口径、扩展验证口径和后续优化工作能够被清晰地区分，避免结果口径漂移。

2.8 架构取舍

本设计优先采用边界清晰、易于验证和便于复现的结构，重点保留已经通过验证矩阵的机制与参数，暂不引入 cache、多发射、乱序等更高风险的复杂结构。这样的取舍使得文中的关键结论能够被实验结果直接支撑，同时也为后续性能优化和实体板验证保留了清晰起点。

3 Verilog/VHDL 建模规范

3.1 建模口径说明

赛题明确要求设计文档包含“Verilog/VHDL 建模规范”。本项目当前实现主体采用 Verilog/SystemVerilog 兼容写法，未单独维护 VHDL 版本 RTL；因此本节按照“与 HDL 语言无关的建模规则 + 当前工程实际采用的 Verilog 写法”两层进行说明，既满足赛题对建模规范的要求，也保证与现有源码完全一致。

3.2 模块划分规则

当前工程遵循“单模块单职责、顶层统一装配”的建模方式。核心模块组织如下：

表 5: RTL 模块划分与职责

模块/文件	主要职责
rtl/YH_rv_cpu.v	CPU 顶层，组织流水级、前递、redirect、CSR 与异常控制
rtl/YH_rv_cpu_if_stage.v	IF 级取指、PC 更新与取指侧控制
rtl/YH_rv_cpu_id_stage.v	ID 级译码、寄存器读、立即数生成与控制准备
rtl/YH_rv_cpu_ex_stage.v	EX 级运算、分支判断、CSR 访问与 trap/mret 决策
rtl/YH_rv_cpu_mem_stage.v	MEM 级 load/store 与 MMIO 访问
rtl/YH_rv_cpu_wb_stage.v	WB 级结果选择与寄存器回写
rtl/YH_rv_cpu_hazard_unit.v	load-use 检测、前递选择与停顿控制
rtl/YH_rv_cpu_decoder.v	指令译码与控制语义拆解
rtl/YH_rv_cpu_regfile.v	32 个通用寄存器实现
rtl/YH_rv_cpu_alu.v	算术逻辑与比较操作
rtl/YH_rv_cpu_soc.v	SoC 封装，连接 ROM、RAM 与 MMIO 外设

这种划分方式对应赛题中“IF/ID/EX/MEM/WB + 核心子模块”的要求，能够保证：

- 模块边界清晰，便于逐模块验证；
- 流水控制与功能单元分离，便于定位冒险与控制流问题；
- CPU、SoC、FPGA 顶层三层边界稳定，便于原型适配与后续扩展。

3.3 组合逻辑与时序逻辑规则

当前工程遵循以下基本建模约束：

1. 组合逻辑优先使用连续赋值或组合 always 块表达，避免隐式锁存器；
2. 时序逻辑统一挂接系统时钟，在同步块内更新寄存器与流水级状态；
3. 流水寄存器、CSR 状态与控制标志只在时序块中更新，保证状态转移单一可追踪；
4. 数据通路宽度与符号扩展在模块边界显式定义，避免跨模块位宽歧义；
5. 冒险处理、redirect 与写回优先级在顶层或专用控制模块统一仲裁，避免分散在多处重复判定。

3.4 参数化与可扩展性规则

赛题要求设计支持参数化与扩展。本项目采用以下规则保证这一点：

- 通过顶层参数控制 `XLEN`，使同一套工程能够完成 RV32/RV64 双 `XLEN` 验证；
- 同步存储器模块与 FPGA 原型顶层通过参数控制 `IMEM_OUTPUT_REG`、`DMEM_OUTPUT_REG` 等关键实现选项；
- 测试平台和构建脚本以目标名、摘要路径和 `manifest` 清单驱动，不把验证路径写死在单个脚本内；
- SoC 层通过固定 MMIO 映射承接 UART、timer、DONE 等外设，为后续扩展更多外设保留接口位置。

3.5 接口与命名规范

为了满足赛题中“模块交互信号列表清晰”的要求，当前工程遵循以下接口规范：

1. 模块命名统一以 `YH_rv_cpu_*` 或 `YH_rv_*` 前缀区分层次与职责；
2. 流水级之间使用显式信号传递数据与控制，不使用隐式全局状态；
3. 存储接口统一区分地址、写使能、写数据和读数据等基本语义；
4. MMIO 访问通过 SoC 层集中解码，CPU 内核不直接感知外设细节；
5. 测试平台命名与 DUT 目标一一对应，例如 `*_tb.v` 用于承接特定场景回归。

3.6 测试平台协同规范

赛题要求建模规范中同时体现 TestBench 框架能力，因此当前工程把测试平台视为建模规范的一部分：

- 模块级 TestBench 用于覆盖 ALU、redirect、hazard 等重点功能单元；
- 系统级 TestBench 统一承接 SoC 固件运行、`riscv-tests` 与 CoreMark；
- 构建脚本负责把软件二进制转换为 ROM 初始化文件，保证软件输入与 RTL 路径一致；
- 结果摘要统一写入 `build/` 目录，便于文档、日志和实现报告之间相互引用。

3.7 本节结论

综合来看，当前工程的 HDL 建模方式已经满足赛题中对“模块化设计、层次化建模、参数化实现、测试平台支撑”的基本要求。说明书在此单列建模规范章节，一方面是为了与赛题交付结构逐项对齐，另一方面也是为了让源码组织方式、验证方法和后续扩展边界能够被评审直接读懂。

4 验证计划与测试报告

4.1 验证矩阵

本设计围绕“功能正确、性能可信、实现可落地”建立了分层验证矩阵：

- Smoke 回归：快速确认脚本、编译链路和最小软件路径可运行；
- Baseline riscv-tests：验证目标 ISA 集合的基本正确性；
- 扩展 full-ui：展示设计在更广测试覆盖下的稳定性；
- CoreMark short 与 strict：评估性能结果并验证长时间运行稳定性；
- Vivado impl50 与 FPGA-like probe：评估 FPGA 原型可实现性和路径一致性。

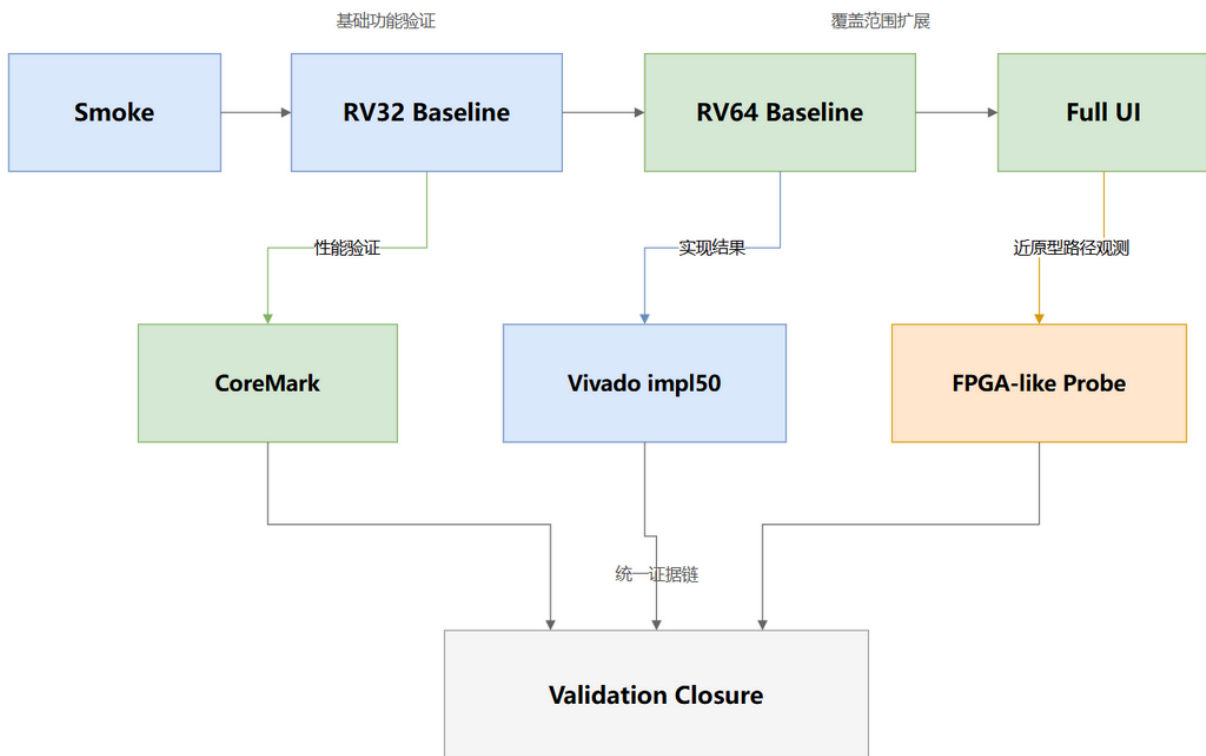


图 3: 验证矩阵与证据闭环示意图

图 3 给出的不是单一测试列表，而是当前工程的验证分层关系：功能正确性负责建立底线，CoreMark 负责给出性能记录，Vivado impl150 和 FPGA-like probe 负责说明原型实现与路径一致性，三者共同构成可以相互印证的证据链。

4.2 验证口径说明

表 6: 验证口径与结果解释

主题	当前口径	说明
冻结比赛 ISA	RV32I + Zicsr	当前性能统计和正式提交主叙述基于该口径
扩展用户态覆盖	rv32i_zicsr_ zifencei / rv64i_ zicsr_zifencei	用于证明更广用户态覆盖下的稳定性， 不等同于冻结比赛 ISA 升级
CoreMark short	快速性能路径	用于日常比较和正式材料中的短跑口径
CoreMark strict	>=10s 长跑路径	用于给出满足 EEMBC 10s 条件的稳定结果
FPGA-like probe	近原型路径观测	用于辅助观察同步原型路径下的相对效率，不替代正式 CoreMark 提交口径

4.3 基础回归与 ISA 验证结果

基础 smoke 回归结果如下：

表 7: Smoke 回归结果

验证项	结果	说明
SoC smoke	PASS	确认最小 SoC、固件和串口输出路径可运行
trap smoke	PASS	确认 trap 进入与异常路径闭环正常
timer irq smoke	PASS	确认 machine timer interrupt 触发与返回路径正常
xlen64 smoke	PASS	确认参数化工程具备基础 XLEN=64 运行能力

基础 ISA 回归结果如下：

表 8: Baseline riscv-tests 结果

验证项	结果	说明
rv32 baseline	33/33	RV32 基本测试集合全部通过
rv64 baseline	21/21	RV64 基本测试集合全部通过

为进一步说明设计在更广测试覆盖下的稳定性，还完成了扩展 full-ui 验证：

表 9: 扩展用户态验证结果

验证项	结果	说明
rv32 full-ui	42/42	覆盖更广的 RV32 用户态测试集合，并验证 fence_i 与 ma_data 等扩展覆盖项
rv64 full-ui	54/54	覆盖更广的 RV64 用户态测试集合，说明参数化工程在更大矩阵下仍保持稳定

4.4 CoreMark 性能结果

CoreMark 采用 “short + strict” 两条路径记录性能。二者使用一致的软件构建与 SoC 运行路径，其中 short 用于快速比较，strict 用于给出满足 $\geq 10s$ 条件的长跑结果。

表 10: CoreMark 结果汇总

路径	Score (Core-Mark/MHz)	周期/时间	说明
CoreMark short	0.912472	11014885 cycles	正式短跑口径，用于快速比较与冻结提交材料引用
CoreMark strict $\geq 10s$	0.912465	1095991523 cycles / 10.959325s	满足 EEMBC 10s 条件的长时间运行结果
FPGA-like probe	7.728811	156442 cycles	用于观察近 FPGA 路径下的相对效率，属于辅助观测口径

short 与 strict 两条路径的 Score 保持一致，说明该设计在长时间运行下没有出现明显性能漂移，也说明其性能结果具有较好的稳定性。当前 CoreMark 报告链路同时保留 short 与 strict 两套摘要，既方便工程迭代，也便于在正式材料中给出可复核的稳定结果。

4.5 FPGA 实现结果

为了支撑 FPGA 原型评估，项目完成了 Vivado impl150 结果收集：

表 11: Vivado impl150 结果

指标	结果	说明
Slice LUTs	2556	综合后逻辑资源占用
Slice Registers	2170	综合后寄存器占用
Block RAM Tile	4	片上存储资源占用
DSP	0	未依赖 DSP 资源
WNS	+5.599ns	50MHz 目标下保持正建立时间裕量
WHS	+0.025ns	保持正保持时间裕量

这组结果说明当前设计不仅能够完成 RTL 仿真和软件运行，还能够在统一实现配置下映射到目标 FPGA 原型平台。对于初赛阶段而言，这一结论的意义在于：方案已经跨过“仅停留在概念设计”的阶段，具备明确的原型实现可行性。

4.6 验证结论

从上述验证矩阵可以得出三点结论：

1. 处理器在 smoke、ISA 回归、CoreMark 与 FPGA 实现结果之间已经形成一致的证据链；
2. RV32/RV64 baseline 与扩展 full-ui 结果表明该设计不仅能完成基础功能，也具备更广验证范围下的稳定基础；
3. 现有结果足以说明该方案具备较高的工程完成度，而实体板卡、最终板级约束与现场观测则被明确保留为后续阶段工作。

5 FPGA 适配指南

5.1 原型目标与实现路径

为说明设计具备工程落地能力，本文进一步给出 FPGA 原型适配路径。当前工作重点是先完成综合、实现、时序、约束和 bring-up 路径整理，在此基础上确认该方案能够稳定映射到目标原型平台，再进入实体板验证。本设计以 Nexys A7-100T 为目标平台，并以 Vivado impl150 流程作为统一实现条件。

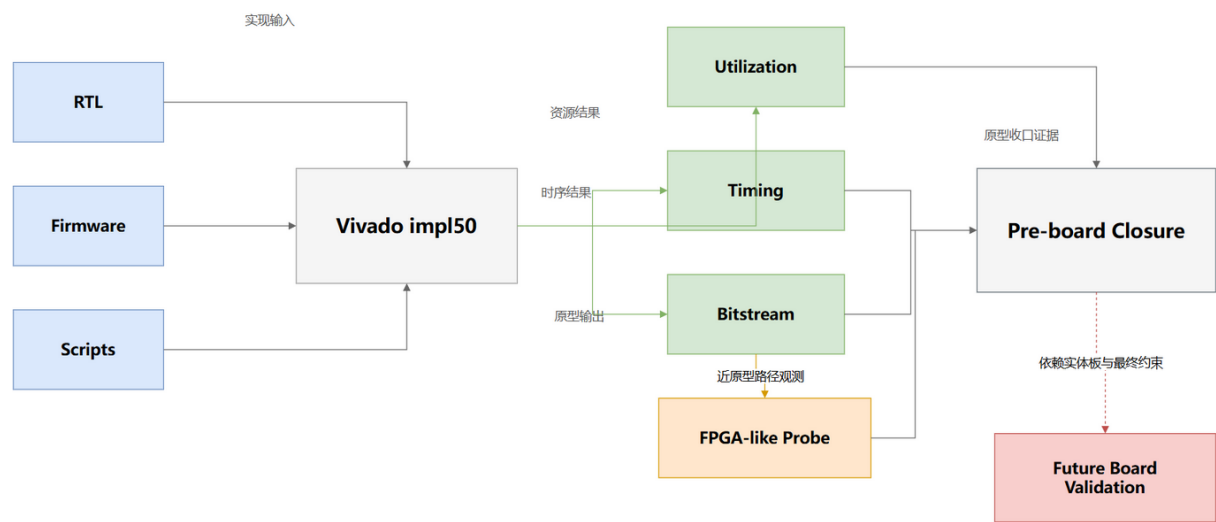


图 4: FPGA 原型适配路径与板级边界示意图

5.2 原型路径固定配置

为了保证报告中的原型结果可复核，当前原型路径已经固定了关键实现条件：

表 12: 原型路径固定配置

配置项	当前值	说明
目标板卡	Nexys A7-100T	当前原型验证与实现报告统一基于该平台
目标流程	Vivado impl50	当前正式原型实现口径固定为 50MHz 路径
IMEM_OUTPUT_REG	0	当前原型路径下的同步取指配置
DMEM_OUTPUT_REG	0	当前原型路径下的数据存储配置
串口口径	115200 8N1	作为后续 bring-up 和软件观测的统一串口口径
报告位置	project/reports/ clk_20p000ns/	统一收纳资源与时序报告，便于复核

5.3 关键结果

表 13: FPGA 原型适配结果

项目	结果	说明
目标平台	Nexys A7-100T	原型验证目标板卡
实现条件	impl50	50MHz 目标频率下的综合实现流程
Slice LUTs	2556	综合实现后的逻辑资源占用
Slice Registers	2170	综合实现后的寄存器资源占用
Block RAM Tile	4	片上存储资源占用
DSP	0	未依赖 DSP 资源
WNS	+5.599ns	50MHz 目标下保持正建立时间裕量
WHS	+0.025ns	保持正保持时间裕量
原型观测周期	156442 cycles	来自 FPGA-like probe 的近板级路径观测
原型观测 Score (Core-Mark/MHz)	7.728811	用于辅助观察原型路径下的相对效率

5.4 实体板相关工作

这里给出的 FPGA 结果属于原型适配阶段结论，主要用于说明设计具备原型实现能力。尚未完成的内容包括 UART/LED 实体连线验证、boot banner 现场观测以及最终板级 I/O 约束确认。这些工作依赖实体板卡与现场条件，因此被保留为后续阶段任务。

5.5 小结

从现有结果看，YH_rv_cpu 已经完成 FPGA 原型适配的关键前置工作，具备继续进入实体板 bring-up 的工程基础。对于初赛阶段而言，这说明该方案不仅完成了 RTL 设计，也具备明确的原型实现路径。换言之，当前结果已经足以支撑“该设计能够落地到目标原型平台”的结论，而剩余板级工作主要属于外部条件驱动下的后续闭环。

5.6 当前可提交的适配材料

按照赛题“FPGA 适配指南”的交付语义，当前说明书已能给出以下适配信息：

- 目标板卡与工具链：Nexys A7-100T + Vivado impl50；
- 固定原型参数：IMEM_OUTPUT_REG=0、DMEM_OUTPUT_REG=0；
- 可复核结果文件：bitstream、利用率报告、时序报告与 FPGA-like probe 摘要；
- 后续 bring-up 入口：UART 串口口径、调试观察点和实体板待完成事项。

这使得本文不仅给出“FPGA 做没做”，还明确说明“如何做、结果在哪里、下一步还差什么”，从而更贴近赛题要求中的“FPGA 适配指南”文档定位。

6 结论与后续工作

6.1 综合结论

YH_rv_cpu 已经形成较完整的初赛工程形态：架构层面采用边界清晰的五级顺序流水，验证层面建立了从 smoke、baseline、扩展覆盖到 CoreMark、impl150 和 FPGA-like probe 的证据链，原型层面完成了可复现的 FPGA 原型适配。综合来看，本设计在初赛阶段具备三项较明确的特点：

- 方案结构清晰，能够较好地说明设计原理与工程取舍；
- 验证矩阵完整，实验结果具备较强的可复核性；
- 资源与时序结果能够支撑后续 FPGA 原型落地，而不是停留在概念设计阶段。

6.2 当前边界

当前说明书中引用的结果已经足以支撑初赛阶段的工程完成度判断，但仍需明确以下边界：

- 性能与正式提交主口径仍以 RV32I + Zicsr 为主，扩展 full-ui 结果用于补充说明更广测试范围下的稳定性；
- FPGA 结果属于 pre-board closure，而不是实体板连线与最终 I/O delay 约束均已完成的最终 signoff；
- 后续若继续开展控制流优化或实现参数调整，仍需在当前验证基线下保持结果和文档同步更新。

6.3 后续工作

后续工作主要包括：

- 在实体板卡到位后完成 UART/LED 连线、boot banner 观测和最终板级 I/O 约束确认；
- 在保持当前验证基线稳定的前提下，继续开展控制流 redirect 成本分析与性能优化实验；
- 结合后续赛程需要，评估是否引入更多扩展指令或进一步增强原型系统能力。

6.4 结语

总体而言，YH_rv_cpu 的价值不在于追求极端复杂结构，而在于以清晰的设计边界、完整的验证矩阵和可复现的实现结果，建立了一套具备工程完整性的处理器方案。

A 关键结果与证据索引

表 14: 关键结果汇总

项目	结果	说明
RV32 baseline	33/33	RV32 基本测试集合
RV64 baseline	21/21	RV64 基本测试集合
RV32 full-ui	42/42	扩展 RV32 用户态覆盖
RV64 full-ui	54/54	扩展 RV64 用户态覆盖
CoreMark short Score (CoreMark/MHz)	0.912472	周期数 11014885
CoreMark strict Score (CoreMark/MHz)	0.912465	周期数 1095991523, 满足 EEMBC 10s 条件
FPGA-like probe Score (CoreMark/MHz)	7.728811	周期数 156442
Vivado impl150	2556 LUT / 2170 FF / 4 BRAM / 0 DSP	综合实现结果
时序结果	WNS = +5.599ns, WHS = +0.025ns	50MHz 目标下保持正裕量

表 15: 关键证据位置

证据类型	主要文件
CoreMark short 摘要	YH_rv_cpu/build/sw/YH_rv_cpu_coremark_rv32_score.summary.txt
CoreMark strict 摘要	YH_rv_cpu/build/sw/YH_rv_cpu_coremark_rv32_strict_2026-04-08.summary.txt
RV32 baseline 摘要	YH_rv_cpu/build/tests/riscv-tests/rv32/summary_baseline_2026-04-08.txt
RV64 baseline 摘要	YH_rv_cpu/build/tests/riscv-tests/rv64/summary_baseline_2026-04-08.txt
RV32 full-ui 摘要	YH_rv_cpu/build/tests/riscv-tests/rv32/summary_ui_all_zifencei_2026-04-08.txt
RV64 full-ui 摘要	YH_rv_cpu/build/tests/riscv-tests/rv64/summary_ui_all_zifencei_2026-04-08.txt
FPGA 资源报告	project/reports/clk_20p000ns/impl_utilization.rpt
FPGA 时序报告	project/reports/clk_20p000ns/impl_timing_summary.rpt
FPGA-like probe 摘要	YH_rv_cpu/build/sw/fpga_probe_i0d0.summary.txt

相关详细日志、实现报告和回归摘要均保留在项目工程中，用于后续复核与继续演进。